Lab1 实验报告 24300160046 章子墨

一、终端中执行 ./check_ops.py bits.c 后的截图

```
$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 24/32 operators
isBetweenEitherOrder: 40/48 operators
mul5Sat: 22/30 operators
classifyAdd3: 24/52 operators
floatScaleThreeHalves: 51/60 operators
floatRoundEven: 45/65 operators
float_i2f: 25/40 operators
bitCount: 25/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

二、终端中执行 ./btest 后的截图

```
$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

三、函数实现思路

P1 signMask

利用左移运算将常量 1 左移 31 位，构造出仅 32 位有符号整数 0x80000000，全程仅使用 1 次移位运算，符合操作数限制。

P2 bitXor

基于德摩根定律（x ^ y = ~(~(x & ~y) & ~(~x & y))）通过与非组合的形式实现 x 与 y 的按位异或运算。

P3 negativePart

利用 32 位有符号整数算术右移的符号扩展特性，将 x 右移 31 位得到全 0 或全 1 的符号掩码；通过补码取反加一计算 x 的相反数，与符号掩码按位与。

P4 copyByteWithin

通过移位运算将指定字节移至最低位，与 255 按位与运算得到单字节值；把这个字节移到 dst 位置；构造目标字节位置的清零掩码，后与 x 相与，就能把 dst 字节清零、保留其余字节；把清空后的 x 与源字节按位或，完成替换。

P5 logicalShift

先对 x 执行算术右移 n 位，再构造一个高 n 位为 0、低 32-n 位全 1 的掩码，与移位结果按位与。

P6 swapNibblePairs

把 0x0F 复制到每个字节的低半字节位置，得到掩码 0x0F0F0F0F。取出每个字节的低 4 位，左移 4 位放到高 4 位；把每个字节的高 4 位右移到低 4 位，再用 m 屏蔽掉多余的位。最后合并。

P7 secondLowestZeroBit

在提取最低为 1 的公式[1]的基础上，利用~x & (x+1)这个公式定位到 x 中最低位 0 的位置并把这个 0 位改成 1；在运用同样的方式找到第二个最低的 0 位。

P8 oddParity

采用二分折叠异或思想[2]，依次将高 16 位、高 8 位、高 4 位、高 2 位、高 1 位与对应低位异或，最后对最低位取反。

P9 rotateRightBits

先对移位位数取 32 的模以保证移位范围有效；提取整数的低 n 位并左移至高位区域，同时将高 32-n 位右移至低位区域，两部分按位或合并，实现 32 位整数的循环右移。先把通过取模运算将 n 规范到 0~31 范围；取出 x 的最低 n 位并左移 32-n 位；其余高位直接右移 n 位，并用掩码清掉高位补进来的符号位；两段运算结果按位或。

---

[1] 提取二进制数最低位 1 的公式参考了该网站：https://blog.csdn.net/u013749051/article/details/134108486

[2] 二分折叠异或法的思想参考了这一网站：https://staaaaaaaaar.github.io/article/rsnfzg0k/#rotateleft-x-n

P10 roundEvenPow2

先拆出商和余数。然后判断是否需要进位，需要进位的情况有：余数大于 2^n 的一半（即 2^n-1），或者余数恰好等于一半且商为奇数。用 carry 表示是否需要进位，
最终结果为商和 carry 的和左移 n 位。

P11 midpointTowardFirst

用无溢出公式求下取整平均[3]；判断 x+y 是否为奇数，如果是奇数说明实际平均值为小数，需要调整；再决定是否需要将结果加 1 进行调整，加 1 的情况是“x+y 为奇数且 x≥y”。

P12 isBetweenEitherOrder

x 在区间内等价于 x 比 a、b 中的一个大，比另一个小，或 x 恰好等于某个端点。分别判断 x 和 a、b 的差值，同时用符号位处理正数 > 负数这类跨符号比较，避免溢出误判；
用异或运算判断 x 值是否等于端点；最后将判断结果综合。

P13 mul5Sat

先计算 x*5 = x + (x<<2)，再通过两种途径检测溢出，溢出时将结果替换为 INT_MAX 或 INT_MIN。用 (x + x_sign) ^ x_sign 求绝对值；若 abs_x >> 29 非零，说明 $|x| \geq 2^{29}$，5|x|必然溢出；计算乘积 prod = x + (x<<2)，比较 x 与 prod 的符号位：符号改变说明加法溢出；合并两种溢出标志，用全 1 / 全 0 掩码做选择：溢出返回 INT_MAX + (符号位)，否则返回 prod。

P14 classifyAdd3

主要思路是把三数和拆成两次加法，分别检测每一步的正 / 负溢出，最后将两次溢出的贡献相加。

P15 floatScaleThreeHalves

将 f × 1.5 转化为定点数 M × 1.5 = M + (M>>1)，按非规范化数和规范化数分别处理，低位按偶数舍入法舍入。

P16 floatRoundEven

根据真指数 exp = e - 127 的范围分档处理，提取整数部分和被舍弃的小数部分，按偶数舍入法舍入后重新编码为浮点数位模式。

P17 float_i2f

定位整数绝对值的最高有效位得到真指数 exp，将去掉最高位后的尾数对齐到 23 位；超过 23 位的低位按偶数舍入法舍入，最后组装符号、偏置指数和尾数。

P18 bitCount

采用分治并行计数，将 32 位逐层分组（2 位→4 位→8 位→16 位→32 位），每组内同时累加 1 的个数，最终得到总数。

P19 bitReverse

采用五层分治交换（16 位→8 位→4 位→2 位→1 位），每层将相邻的位块左右互换，最

[3] 无溢出求平均值的公式参考了该网站：https://zhuanlan.zhihu.com/p/2016871218825961744

终完成全位反转。构造五个掩码的过程中运用到了异或链[4]的思想。

[4] 异或链的思想参考了该网站：https://graphics.stanford.edu/~seander/bithacks.html